KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 1 Laboratory Sheet

Constraint Satisfaction Problems: From a Paper Model to a Running Solver

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours

Step 10 steps · 4 checkpoints

Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), Python 3.12, a text editor, and network access for a one-time package install

1 2 3 4 ✓ CP1 5 6 ✓ CP2 7 8 ✓ CP3 9 10 ✓ CP4

Learning phases: 1 · Pre-instruction | 2 · Content | 3 · Participation | 4 · Assessment | 5 · Follow-through

Why It Matters 1 · Pre-instruction

In the next two hours you will move from a sentence on paper to a program that actually finds — and counts — solutions. Every classic CSP you touch this semester starts exactly the way today's scripts start: a handful of variables, a domain, one or two constraints, and a single call to a solver. Today you build and run that pattern with your own hands, on four increasingly real problems, and close by writing one from scratch.

Prior Knowledge 1 · Pre-instruction

This sheet exercises all four units of this week's chapter: the formal CSP triple (Unit 1), the modeling choices behind a good model (Unit 2), the paper formulations of N-queens, Sudoku and map coloring (Unit 3), and their OR-Tools CP-SAT code (Unit 4). Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: The Course's Shared Python Environment

Step 1: Create and activate the virtual environment

What will you do? You will create the course's shared Python 3.12 virtual environment at `kod_ornekleri/.venv` and install the two packages this week's code needs, at the exact pinned versions the whole course uses.

Why? Every model in this course is checked against a real, captured solver run; pinning `ortools` and `pytest` to fixed versions is what makes those captured outputs reproducible on your own machine.

Terminal

```
$ cd kod_ornekleri
$ python3.12 -m venv .venv
$ source .venv/bin/activate
$ pip install ortools==9.14.6206 pytest==9.1.1
```

Expected result: `pip` downloads and installs a handful of packages and finishes with a line naming every package it installed, including `ortools` and `pytest` at exactly the versions above.

Expected Output

```
Collecting ortools==9.14.6206
  Using cached ortools-9.14.6206-cp312-cp312-macosx_11_0_arm64.whl (20.2 MB)
Collecting pytest==9.1.1
  Using cached pytest-9.1.1-py3-none-any.whl (386 kB)
...
Installing collected packages: typing-extensions, six, pygments, protobuf,
pluggy, packaging, numpy, iniconfig, immutabledict, absl-py,
python-dateutil, pytest, pandas, ortools
Successfully installed absl-py-2.5.0 immutabledict-4.3.1 iniconfig-2.3.0
numpy-2.5.3 ortools-9.14.6206 packaging-26.3 pandas-3.0.5 pluggy-1.6.0
protobuf-6.31.1 pygments-2.21.0 pytest-9.1.1 python-dateutil-2.9.0.post0
six-1.17.0 typing-extensions-4.16.0
```

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead. If script execution is disabled, run PowerShell as administrator once and allow it with the command below.

Terminal

```
$ Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Step 2: **Verify the installed versions**

What will you do? With the environment still active, you will print the installed `ortools` version and the `pytest` version to confirm the pinned install succeeded.

Why? A silent, successful `pip install` does not by itself prove the right version landed on your `PATH` — especially if you have another Python installation elsewhere on your machine; this check catches that mistake before it costs you an hour later in the sheet.

Terminal

```
$ python -c "import ortools; print(ortools.__version__)"
$ python -m pytest --version
```

Expected result: The first command prints `9.14.6206`; the second prints a line starting with `pytest 9.1.1`.

Expected Output

```
9.14.6206
pytest 9.1.1
```

Common Pitfall

If the first command instead raises `ModuleNotFoundError: No module named 'ortools'`, your virtual environment is almost certainly not active — re-run the `source/Activate.ps1` command from the previous step before trying again.

2 Unit 1 — The Formal CSP Triple

(See Course Notes Week 1, Unit 1 — What a CSP Is.)

Step 3: Identify the CSP components in a paper scenario

What will you do? Read the scenario below and, on paper, write down the variables, one plausible domain, and the one constraint it implies; then say whether finding *any* valid assignment makes this a CSP or a COP.

"This week, three research assistants — RA1, RA2 and RA3 — each need exclusive use of one of two shared machines: the department's GPU workstation or its CPU cluster login node. RA1 and RA2 are running experiments that would corrupt each other's results if run on the same machine at the same time, so they must not be assigned the same machine."

Why? This is the same three-part exercise Unit 1's practice task used on a talk-scheduling scenario, applied here to a fresh, lab-resource scenario — writing out the triple for a second, unrelated situation is what turns being able to follow this definition into being able to apply it independently.

Expected result: Variables: one per assistant (RA1, RA2, RA3). Domain: {GPU, CPU} for each. Constraint: value(RA1) ≠ value(RA2). As stated — any assignment respecting that one constraint is acceptable — this is a pure CSP; it would only become a COP if an objective were added on top, for example "minimize the number of assistants waiting for the GPU."

Step 4: Run the toy triangle CSP and connect its output back to the triple

What will you do? You will run `csp_basics.py`, the tiny 3-region "triangle" toy CSP from Unit 1's worked example, and match each part of its printed output back to the formal triple.

Why? Unit 1 builds this exact model by hand, on paper, before any code exists; running it here is your first proof that a paper triple (variables, domains, constraints) and a working CP-SAT program are two views of the exact same object.

The variables — three regions, domain $\{0, 1, 2\}$ (three colours):

```
12 region_a = model.NewIntVar(0, 2, "region_a")
13 region_b = model.NewIntVar(0, 2, "region_b")
14 region_c = model.NewIntVar(0, 2, "region_c")
15 return [region_a, region_b, region_c]
```

The constraints — one pairwise inequality per edge of the triangle:

```
22 model.Add(region_a != region_b)
23 model.Add(region_b != region_c)
24 model.Add(region_a != region_c)
```

Terminal

```
$ cd hafta01
$ python modeller/csp_basics.py
```

Expected result: A status of `OPTIMAL` and one valid three-colouring of the triangle, with all three regions pairwise distinct — exactly what Unit 1's three constraints require.

Expected Output

```
Status: OPTIMAL
Assignment: region_a=2, region_b=1, region_c=0
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta01` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta01`.

Checkpoint — verify before moving on
You have written a CSP triple entirely on paper and watched CP-SAT solve one from Unit 1's worked example, with the printed assignment satisfying every constraint you would have checked by hand. If both match what the `Expected result` boxes above describe, move on to Unit 2.

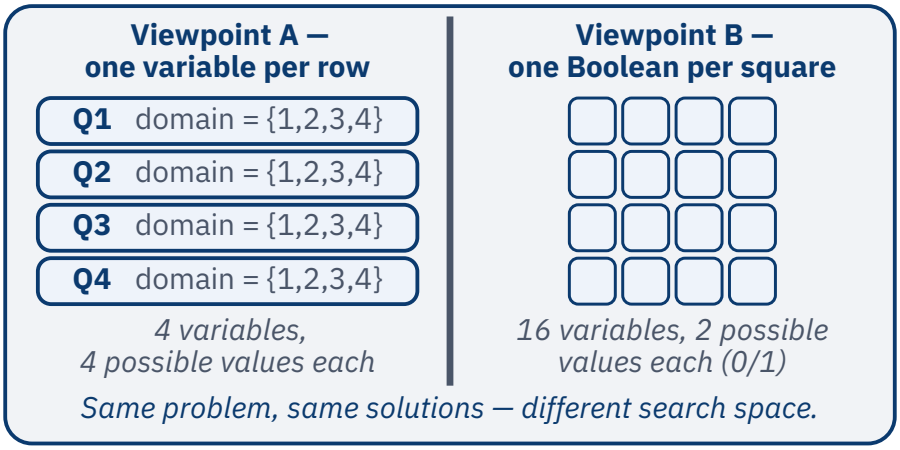
3 Unit 2 — Modeling as a Skill

(See Course Notes Week 1, Unit 2 — Modeling as a Skill.)

Step 5: Compare two viewpoints for 4-queens, then propose an auxiliary variable

What will you do? Study the figure below, which places Unit 2's two 4-queens viewpoints side by side. For each viewpoint, compute the number of variables and the size of each variable's domain, then multiply them to get the raw number of candidate assignments *before any constraint is applied*. Second, propose one *auxiliary* variable you could add to the map-coloring model (one variable per region, domain = 3 colours) to make the question "how many distinct colours does this solution actually use" easy to check, without changing what counts as a solved map.

Why? Unit 2's whole point is that two correct models of the same problem are not equally good models; counting raw candidate assignments for both 4-queens viewpoints turns "one is more compact" from a qualitative impression into a number you computed yourself, and the auxiliary-variable question rehearses the same lever (Unit 2's practice task used it on Sudoku) on a problem you will code in Unit 4.



Expected result: Viewpoint A (one queen per row): 4 variables, domain size 4 each, so $4^4 = 256$ raw candidate assignments. Viewpoint B (one Boolean per square): 16 variables, domain size 2 each, so $2^{16} = 65536$ raw candidate assignments — 256 times more, before either model's constraints even start ruling anything out. For the auxiliary variable, one reasonable answer is a per-colour Boolean `used[colour]`, forced equal to "at least one region is assigned this colour"; the original per-region colour variables still fully determine a solution, `used[colour]` just gives the "how many colours are actually used" question a direct handle instead of re-scanning every region variable each time.

4 Unit 3 — Classic Problems as Models

(See Course Notes Week 1, Unit 3 — Classic Problems as Models.)

Step 6: Formulate a small Sudoku on paper, then hand-trace a candidate solution

What will you do? Below is a 4×4 Sudoku puzzle (values 1–4, 2×2 boxes instead of 3×3). First, state its full CSP triple: the variables, their domain, and every constraint, including the givens — this is Unit 3's Sudoku formulation, scaled down for hand-checking. Second, check the candidate grid that follows it against *every* row, column and box constraint, and report whether it is a valid solution; if not, name a constraint it violates.

1	.	.	.
.	.	1	2
.	3	.	.
.	.	.	4

1	4	2	3
3	4	1	2
4	3	2	1
2	1	3	4

(Left: the puzzle's givens, . for empty. Right: a classmate's candidate solution to check.)

Why? This week deliberately stays out of propagation (Week 2's subject), so "hand-trace" here means checking a *complete* candidate assignment against every constraint — exactly the "solution" half of Unit 1's definition — not searching for one. Doing this once by hand is what makes "AllDifferent over every row, column and box" in Unit 4's Sudoku code feel like a direct translation rather than a leap of faith.

Expected result: Triple: one variable per cell (16 variables), domain $\{1, 2, 3, 4\}$ each; constraints are the five givens fixed by equality, plus `AllDifferent` over every row, every column, and each of the four 2×2 boxes. Checking the candidate: row 1 reads $\{1, 4, 2, 3\}$ (fine on its own), but column 2 reads $\{4, 4, 3, 1\}$ — the value 4 appears twice (row 1 and row 2) — so this is *not* a valid solution. The same single swap also breaks the top-left 2×2 box, which reads $\{1, 4, 3, 4\}$.

Common Pitfall

A single misplaced pair of cells often breaks more than one constraint at once, because rows, columns and boxes overlap on every cell. Do not stop checking after you find the first violated row — a full hand-trace checks all three constraint families, since a grader (or an automated test) will.

Checkpoint — verify before moving on

You have quantified why one 4-queens viewpoint is more compact than the other, proposed a working auxiliary variable for map coloring, stated a full Sudoku triple on paper, and correctly flagged the candidate grid above as invalid with a specific violated constraint. If all four hold, you are ready to see these paper models run as code.

5 Unit 4 — First Models in OR-Tools CP-SAT

(See Course Notes Week 1, Unit 4 — First Models in OR-Tools CP-SAT.)

Step 7: Run the N-queens model: one solution, then every solution

What will you do? You will run `nqueens.py`, which solves 8-queens once and then enumerates every solution for 6-queens, and check both printed results against the real captured output below.

Why? This is Unit 3's "one queen per row" paper model, unchanged, turned into three `AddAllDifferent` calls; running it here closes the loop Unit 4 walks through — from a formulation you already checked by hand to a program whose output you can verify against a known fact (6-queens has exactly 4 solutions, OEIS A000170).

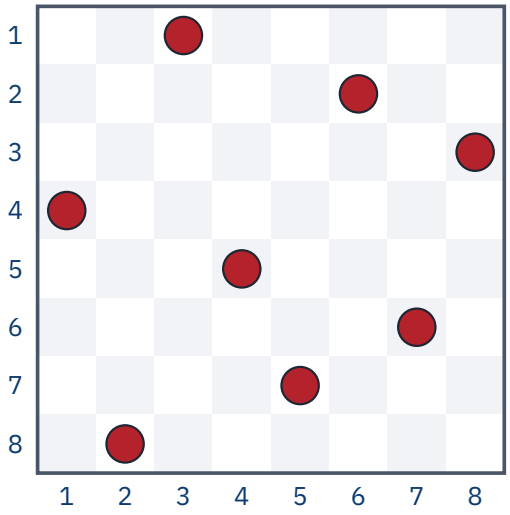
Terminal

```
$ python modeller/nqueens.py
```

Expected result: One valid 8-queens board, printed as a column list, followed by the exact count of 6-queens solutions.

Expected Output

```
Status: OPTIMAL
Columns per row (0-indexed) for N=8: [2, 5, 7, 0, 3, 6, 4, 1]
Number of solutions for N=6: 4
```



One of 92 solutions for 8-queens (Week 1); see Week 3 for how search finds it and Week 2 for how propagation prunes it.

Expected result: The figure above draws this exact solution: row 1's queen sits at 0-indexed column 2, drawn as 1-indexed column 3 on the board, and so on down the printed list — one shift of 1 applied uniformly to every entry.

Step 8: Run Sudoku and map coloring, then run the week's automated test gate

What will you do? You will run `sudoku.py` and `map_coloring.py` and check their output, then run the full `pytest` gate for this week's code and confirm every test passes.

Why? `sudoku.py` and `map_coloring.py` are the same direct translations Unit 4 walks through — 81 cell variables with `AllDifferent` per row/column/box for Sudoku, one pairwise `!=` per adjacency edge (no `AllDifferent`) for map coloring. The test gate is this week's independent check that every model's solution is not just *a* solution CP-SAT accepted, but a *correct* one by the classic problem's own rules — the same kind of check you just did by hand on the Sudoku grid in Unit 3's step.

Terminal

```
$ python modeller/sudoku.py
$ python modeller/map_coloring.py
$ python -m pytest -q
```

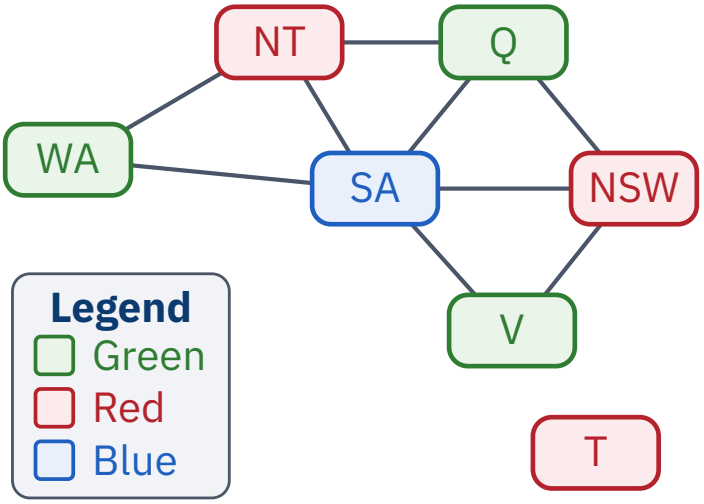
Expected result: Sudoku prints a fully completed 9×9 grid; map coloring prints one colour per region with `T` (Tasmania) coloured despite bordering no other region; `pytest` reports every test passing.

Expected Output

```
Status: OPTIMAL
WA: Green
NT: Red
SA: Blue
Q: Green
NSW: Red
V: Green
T: Red
```

Expected Output

```
.....                                                                    [100%]
5 passed in 0.24s
```



Checkpoint — verify before moving on
All three of Unit 3's paper models — N-queens, Sudoku, map coloring — have now produced verified CP-SAT output, and the week's full `pytest` gate passes on your own machine. You are ready for the one model this sheet has not shown you yet.

6 Capstone — Build a Complete Model From Scratch

Step 9: Run the incomplete SEND + MORE = MONEY starter

What will you do? `SEND + MORE = MONEY` is a classic constraint puzzle: assign each of the eight letters S, E, N, D, M, O, R, Y a distinct digit 0–9 (S and M may not be 0, since they lead a number) so that the sum, read as ordinary arithmetic on the digits each letter stands for, holds true. A starter file, `kod_ornekleri/hafta01/lab/send_more_money.py`, already declares the eight digit variables and the distinctness/leading-digit constraints shown below; run it once, exactly as given.

Why? Running the model before it is finished is deliberate: seeing what `AllDifferent` plus the two leading-digit constraints alone produce — *some* distinct-digit assignment, almost certainly not the puzzle's answer — makes it concrete that the arithmetic itself is not yet part of the model at all.

The starter's variables — one integer 0–9 per letter:

```
15 digit = {letter: model.NewIntVar(0, 9, letter) for letter in LETTERS}
```

Its constraints so far — pairwise distinctness plus the two leading-digit rules:

```
19 model.AddAllDifferent(digit.values())
20 model.Add(digit["S"] != 0)
21 model.Add(digit["M"] != 0)
```

Terminal

```
$ python lab/send_more_money.py
```

Expected result: A status of `OPTIMAL` and *some* assignment of digits 0–9 to the eight letters — but almost certainly *not* one that makes `SEND + MORE` equal `MONEY`, since the model so far only knows the letters must be pairwise distinct.

Expected Output

```
Status: OPTIMAL
Assignment: {'S': 2, 'E': 7, 'N': 6, 'D': 4, 'M': 5, 'O': 3, 'R': 1, 'Y': 0}
```

Step 10: **Add It Yourself: complete the model and verify the unique solution**

What will you do? Add the one constraint the starter is missing. Treat each four- or five-letter word as a number built from its digits' positional values, exactly the way `sudoku.py`'s givens are applied by equality in Unit 4 — for example `SEND` is $1000{\cdot}S + 100{\cdot}E + 10{\cdot}N + D$. Write one `model.Add(...)` call stating `SEND + MORE == MONEY` as a single linear equation over the eight `digit[...]` variables, add it where the file's `TODO` comment says to, then re-run the script.
Why? This is Unit 4's three-piece recipe — model, variables, constraints, solve — applied without a paper model handed to you first; completing it yourself is this week's integrative step, the one that closes the loop from a formal definition (Unit 1) all the way to a verified, running solver call.

Terminal

```
$ python lab/send_more_money.py
```

Expected result: A unique solution: `S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2` — i.e. $9567 + 1085 = 10652$. This is the puzzle's one and only solution (a well-known fact about this classic puzzle); if your printed digits differ from this, re-check your equation's positional weights before assuming your distinctness constraints are at fault.

Expected Output

```
Status: OPTIMAL
Assignment: {'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
```

Tip

Build the three word-sums as ordinary Python expressions first (e.g. `send = 1000 * digit["S"] + 100 * digit["E"] + 10 * digit["N"] + digit["D"]`), and only then wrap the whole comparison in a single `model.Add(...)` — this mirrors how `nqueens.py` builds a Python list comprehension *before* passing it to `AddAllDifferent`.

Common Pitfall

Writing `send + more == money` on its own line, outside `model.Add(...)`, is Unit 4's classic mistake repeated one level up: Python happily compares the `IntVar` objects themselves and throws the boolean result away — the solver never sees this constraint, and your model still only enforces distinctness.

Checkpoint — verify before moving on

Your completed `send_more_money.py` reports exactly one solution, and its eight digits match `S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2`. This is the same variables-domains-constraints-solve pattern you have now built four times today, the last time entirely from a written description rather than a paper model someone else formulated for you.

Troubleshooting

Symptom	Likely cause	Fix
`python3.12: command not found`	Python 3.12 is not installed, or is installed under a different command name	Try `python3 --version` to check what is available; install Python 3.12 if it is missing
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`
`pip install` reports a version conflict	An older `ortools` or `pytest` is already installed globally and being reused	Confirm the virtual environment is active (prompt shows `(.venv)`), then re-run the install command
`python modeller/nqueens.py` prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta01` before running any script
Your `send_more_money.py` run reports `INFEASIBLE`	The positional-value equation was built with the wrong digit weights (e.g. tens and hundreds swapped)	Re-derive `send`, `more` and `money` one word at a time, checking each digit's place value against the word's letters left to right
`pytest -q` reports failures unrelated to your own edits	`modeller/` files were edited directly instead of the `lab/` starter file	Restore `modeller/*.py` from the course repository; only `lab/send_more_money.py` is meant to be edited this week

Transfer — Where Else? 5 · Follow-through

The four scripts you ran and completed today are not disposable lab work: Week 2 opens the solver's black box using the exact same `nqueens.py` and `map_coloring.py` files you just ran, Week 3 asks which variable and value order finds a solution fastest on these same models, and Week 6 runs the same N-queens/Sudoku/map-coloring models through two more solvers entirely — MiniZinc's Gecode and COIN-BC/HiGHS backends. Keep every file in `kod_ornekleri/hafta01` exactly where it is. More generally, the pattern you practiced eight times today — state variables, a domain and constraints, then hand the whole thing to a solver rather than write your own search — is the single habit this entire course exists to

build; whatever combinatorial problem you meet next, in a paper, in your thesis, or in a future job, the first useful question is almost always “what are this problem’s variables, domains and constraints,” asked in exactly that order.